

TP 5 — Industrialisation d'un pipeline Airflow Open-Meteo

Contexte

Vous devez faire évoluer un pipeline Airflow existant autour de l'API Open-Meteo.

Le pipeline devra récupérer des données météo, les archiver, les transformer, contrôler leur qualité, décider si elles peuvent être chargées, puis tracer le résultat de l'exécution.

L'objectif est de produire un pipeline Airflow robuste, lisible, maintenable, observable et relançable sans incohérence.

Objectifs du TP

Vous devez mettre en œuvre les notions suivantes :

- orchestration avec Apache Airflow ;
- extraction depuis une API externe ;
- archivage de données brutes ;
- transformation de données ;
- chargement dans PostgreSQL ;
- contrôle qualité ;
- branchement conditionnel ;
- gestion des erreurs ;
- retries, retry delay et timeout ;
- idempotence ;
- logs applicatifs ;
- traçabilité d'ingestion ;
- conventions de nommage ;

- structuration propre d'un projet Airflow.

Sujet

Vous devez construire un pipeline Airflow complet autour des données météo Open-Meteo.

Le pipeline doit traiter plusieurs villes configurables et charger uniquement les données considérées comme valides.

En cas d'anomalie qualité, le pipeline ne doit pas charger les données finales et doit tracer l'anomalie.

Cas à démontrer

Vous devez démontrer les cas suivants.

Cas nominal

Le pipeline doit récupérer, archiver, transformer, valider et charger les données.

Vous devez fournir une preuve d'exécution réussie.

Cas d'anomalie qualité

Vous devez simuler une anomalie qualité.

Le pipeline doit détecter l'anomalie, empêcher le chargement final et tracer le problème.

Vous devez fournir une preuve d'exécution de ce cas.

Cas de relance

Vous devez relancer le DAG ou une partie du pipeline.

La relance ne doit pas créer de doublons.

Vous devez fournir une preuve de l'idempotence.

Livrables attendus

Vous devez rendre :

```
code complet du DAG
modules Python séparés
scripts SQL nécessaires
README
schéma logique du pipeline
preuve d'exécution nominale
preuve d'exécution avec anomalie qualité
preuve de relance sans doublon
preuve des logs Airflow
preuve du contenu des tables PostgreSQL
```

Contenu du README

Le README doit contenir :

```
description du pipeline
schéma du workflow
liste des variables Airflow utilisées
liste des connexions Airflow utilisées
description des tâches du DAG
stratégie de robustesse
stratégie d'idempotence
contrôles qualité mis en place
règle de branchement conditionnel
description des logs produits
description des tables PostgreSQL
preuves d'exécution
limites éventuelles du travail rendu
```